Canvas extensions

Define the contract, state, and allowed actions

Today's outcome

Build one delivery-readiness canvas in the Copilot app.

Record:

- the contract;
- one direct update;
- one agent-requested update;
- one rejected update and its recovery.

You need the Copilot app and the built-in `/create-canvas` skill. If project scope is unavailable, use user scope.

Choose the right customization

Need	Use
Repository convention	Instructions
Repeatable procedure	Skill
Specialist behavior	Custom agent
External tool or data	MCP server
Shared interactive artifact	Canvas
Packaged customizations	Plugin

Use a canvas when people and an agent need to inspect or change the same visible state.

Compare with M3-03, M3-04, and M3-05: a skill gives know-how, a custom agent gives a role, an MCP server gives reach. A canvas gives a shared surface.

Terms

Term	Meaning
Canvas extension	A shared, interactive panel in the Copilot app for one work artifact: a board, plan, checklist, or dashboard
Agent-callable capability	A named action the agent can call on the canvas, such as `move_card`. Each has an input schema
Human action	A control in the panel: button, card, or filter
Plugin	A package that can ship a canvas with skills or MCP servers. You install it from **Customize > Canvas**

People and the agent change the **same state**. Capabilities are generated from your prompt, and you can ask the agent to add, remove, or revise them.

Boundaries

A canvas is	A canvas is not
A visible, shared working surface	The system of record
Limited to the capabilities you list	A way to give the agent broad access
Editable by a person and the agent	A replacement for review

A capability is the agent's whole reach. If the contract does not list it, remove it. Dependencies and persisted data come with the extension, so read them before you keep it.

Canvas anatomy

```
shared state
  ├─ human actions in the interface
  ├─ agent-callable capabilities
  └─ validation before every change
```

The **contract** is a short document you write first. It names all four parts. Copilot generates the extension from the contract, and you check the result against it.

Generated files

Scope decides where the extension lives:

Scope	Path
Project (repository)	`.github/extensions/<name>/`
User	`~/.copilot/extensions/<name>/`

Docs describe a typical layout. **Check what Copilot actually generated.**

- `package.json`: metadata and dependencies
- an entry file such as `extension.mjs`: canvas behavior, capabilities, validation
- optional JSON under `artifacts/`: persisted state

Read the entry file against the contract. Check `package.json` for packages you did not ask for.

Worked contract

Part	Delivery-readiness rule
State	item, status, review flag, evidence
Status	`planned`, `in-progress`, `review`, `ready`
Human actions	set status, flag review, add evidence
Agent capabilities	read board, update one named item, list flagged items
Validation	known item, accepted status, evidence before `ready`

Keep authority narrow

Allowed:

- read the current board;
- update one named item;
- list items flagged for review.

Out of scope:

- deployment;
- external writes;
- credentials;
- hidden data imports;
- broad actions such as "manage the project."

Prepared demo inputs

ID	Item	Start state	Evidence
DOC-101	Draft operator guide	`planned`	none
API-204	Add retry metric	`in-progress`	test log
WEB-318	Check empty state	`review`	preview

Demo sequence

1. Optional: run the local scaffolding tests (they check the exercise contract, not Copilot).
2. Create the canvas from the worked prompt.
3. Move `WEB-318` to `ready` with a visible control.
4. Ask the agent to move `API-204` to `review`.
5. Ask for `DOC-101` to move to `ready`.
6. Show the rejection, add evidence, and retry.

Creation prompt

```
/create-canvas

Create a delivery-readiness canvas from @prepared-items.json.
Use the contract in @canvas-contract.md.
People can add evidence and update one item.
Copilot can read the board, update one named item, and list review flags.
Reject unknown items or statuses. Require evidence before ready.
Do not connect external systems.
```

Use project scope when `.github/extensions` is allowed. Otherwise use user scope.

Two paths, one state

Path	Result
Human control	The item visibly changes in the canvas
Agent capability	The named item changes and the response matches the canvas

If the response and visible state disagree, stop and inspect the artifact.

Rejection is part of the design

Request:

```
Move DOC-101 to ready.
```

Expected:

```
Rejected: evidence is required before ready.
```

The state stays unchanged and the message names the rule.

Recovery keeps the rule

1. Add synthetic evidence to `DOC-101`.
2. Retry the same transition.
3. Check that `ready` is now accepted.
4. Record the state before, the rejection, the correction, and the state after.

Review the generated extension

Check:

- state fields and defaults;
- visible actions;
- agent-callable capabilities;
- dependencies and persisted data.

Remove anything the contract did not request.

About the local tests

`lab/starter/src/canvas-contract.mjs` and its tests are **hand-built scaffolding**. They encode this exercise's rules in plain JavaScript.

- They do **not** run, test, or imitate Copilot canvas behavior.
- They give you a fast check of the rules before you generate anything.
- The real check is the generated canvas against the contract.

Pros, cons, and limits

Pros	Cons and limits
Visible state you can inspect and steer	Canvases are new. Behavior and docs may change
People and agent share one artifact	Generated code needs review like any code
State survives turns and handoffs	A canvas is not a system of record
Capabilities limit what the agent can call	Needs the Copilot app. Project scope needs write access to `.github/extensions`

Stay in chat for a short answer or one draft. Check the current docs before teaching.

Source of truth

The canvas is a working artifact, not a system of record.

If another system owns the data, label the canvas as a projection and keep external writes out of scope.

Lab handoff

Use one contract and one prepared data set for the full two hours.